

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №8
« Мережеве та асинхронне програмування. REST API »
з курсу «Професійний Python»
Варіант №5 — « Каталог автомобілів »**

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 06.10.2026

Львів — 2026

Лабораторна робота №8 | Варіант №5

1. Мета роботи

Сформувати практичні навички розробки сучасних веб-додатків та RESTful API на базі фреймворку FastAPI, використання реляційних баз даних через SQLAlchemy, валідації даних за допомогою Pydantic, а також реалізації асинхронних мережових запитів із застосуванням бібліотеки `httpx`, механізмів паралелізму (`asyncio.Task`, `asyncio.gather`, `asyncio.Semaphore`), тайм-аутів та повторних спроб (`retry`) у Python.

У результаті виконання лабораторної роботи студент повинен уміти створювати REST API ендпоінти, проєктувати схеми валідації вхідних та вихідних даних, інтегрувати бізнес-логіку з шаром зберігання даних (ORM), виконувати асинхронні HTTP-запити, керувати обмеженням паралелізму за допомогою семафорів, обробляти винятки мережі та писати автоматизовані інтеграційні тести.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Каталог автомобілів». Для кожного автомобіля зберігати ідентифікатор (`id`), марку (`make`), модель (`model`), рік випуску (`year`), ціну (`price`), пробіг (`mileage`), VIN-код (`vin`) та зв'язок із виробником (`manufacturer_id`). Реалізувати повноцінний REST API сервер для управління каталогом автомобілів та виробників за допомогою фреймворку FastAPI із підключенням реляційної бази даних через SQLAlchemy.

Проєкт повинен мати рекомендовану структуру (`schemas.py`, `models.py`, `repositories.py`, `api.py`, `external_api.py`, `test_api.py`), використовувати Pydantic-моделі для валідації створення (`CarCreate`), оновлення (`CarUpdate`) та серіалізації відповідей (`CarResponse`), маршрути FastAPI (`GET`, `POST`, `PATCH`, `DELETE`) для ресурсів автомобілів та фільтрації їх за виробниками, асинхронні функції взаємодії з зовнішніми сервісами через `httpx.AsyncClient` з використанням `asyncio.create_task`, `asyncio.gather`, обмеження потоків через `asyncio.Semaphore`, а також механізми обробки помилок, тайм-аутів і повторних спроб (`retry`), а також автоматизовані інтеграційні тести з використанням `TestClient` для перевірки кодів відповідей, валідації та коректності виконання CRUD-операцій.

3. Структура проєкту

```
Lab08/
├─ pyproject.toml
├─ README.md
├─ .gitignore
├─ report/
│   └─ Хілінська_Іванна_Ла608_V5.pdf
├─ tests/
│   └─ conftest.py
```

```

|   └─ test_repositories.py
|   └─ test_api.py
└─ src/
    └─ car_catalog/
        ├── __init__.py
        ├── main.py
        ├── api.py
        ├── schemas.py
        ├── database.py
        ├── models.py
        ├── repositories.py
        ├── services.py
        └─ external_api.py

```

Каталог `src/car_catalog` містить пакет застосунку для побудови REST API та роботи з базою даних каталогу автомобілів відповідно до рекомендованої структури проєкту. Архітектура включає головний файл запуску (`main.py`), модуль маршрутизації API (`api.py`), схеми валідації даних (`schemas.py`), налаштування підключення до бази даних (`database.py`), моделі даних ORM (`models.py`), репозиторії для роботи з даними (`repositories.py`), сервісний шар (`services.py`) та асинхронний мережевий шар (`external_api.py`). Тести розміщені у папці `tests/` (`test_repositories.py`, `test_api.py` та конфігурація `conftest.py`), а звітні матеріали — у папці `report/`.

4. Опис створених модулів

4.1. `schemas.py`

Pydantic-схеми для валідації вхідних даних (Create/Update) та серіалізації відповідей.

```
from pydantic import BaseModel, Field
```

```

class CarCreate(BaseModel):
    make: str = Field(min_length=1, max_length=100)
    model: str = Field(min_length=1, max_length=100)
    year: int = Field(ge=1886, le=2100)
    price: float = Field(ge=0)
    mileage: int = Field(ge=0)
    vin: str | None = Field(default=None, max_length=17)
    manufacturer_id: int | None = Field(default=None, gt=0)

```

```

class CarUpdate(BaseModel):

    make: str | None = Field(default=None, min_length=1, max_length=100)

    model: str | None = Field(default=None, min_length=1, max_length=100)

    year: int | None = Field(default=None, ge=1886, le=2100)

    price: float | None = Field(default=None, ge=0)

    mileage: int | None = Field(default=None, ge=0)

    vin: str | None = Field(default=None, max_length=17)

    manufacturer_id: int | None = Field(default=None, gt=0)


class CarResponse(BaseModel):

    id: int

    make: str

    model: str

    year: int

    price: float

    mileage: int

    vin: str | None

    manufacturer_id: int | None

    model_config = {

        "from_attributes": True

    }

```

4.2. api.py

Головний додаток FastAPI з усіма необхідними ендпоінтами.

```

from fastapi import Depends, FastAPI, HTTPException, status

from sqlalchemy.orm import Session

from src.car_catalog.database import SessionLocal, Base, engine

from src.car_catalog.repositories import (

    CarRepository,

    CarDB,

    ManufacturerDB,

```

```

)

from src.car_catalog.schemas import CarCreate, CarResponse, CarUpdate

app = FastAPI(
    title="Car Catalog API",
    version="1.0.0",
)

Base.metadata.create_all(bind=engine)

def get_session():
    session = SessionLocal()
    try:
        yield session
    finally:
        session.close()

@app.get("/cars", response_model=list[CarResponse])
async def get_cars(session: Session = Depends(get_session)):
    repo = CarRepository(session)
    return repo.list_all()

@app.get("/cars/{car_id}", response_model=CarResponse)
async def get_car(car_id: int, session: Session = Depends(get_session)):
    repo = CarRepository(session)
    car = repo.get_by_id(car_id)
    if not car:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Car not found")
    return car

@app.post("/cars", response_model=CarResponse, status_code=status.HTTP_201_CREATED)
async def create_car(data: CarCreate, session: Session = Depends(get_session)):
    repo = CarRepository(session)
    car = repo.create(

```

```

        make=data.make,

        model=data.model,

        year=data.year,

        price=data.price,

        mileage=data.mileage,

        vin=data.vin,

        manufacturer_id=data.manufacturer_id,

    )

    return car

@app.patch("/cars/{car_id}", response_model=CarResponse)
async def update_car(car_id: int, data: CarUpdate, session: Session = Depends(get_session)):
    repo = CarRepository(session)

    car = repo.get_by_id(car_id)

    if not car:
        raise HTTPException(status_code=404, detail="Car not found")

    updates = data.model_dump(exclude_unset=True)

    for field, value in updates.items():
        setattr(car, field, value)

    session.commit()

    session.refresh(car)

    return car

@app.delete("/cars/{car_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_car(car_id: int, session: Session = Depends(get_session)):
    repo = CarRepository(session)

    success = repo.delete(car_id)

    if not success:
        raise HTTPException(status_code=404, detail="Car not found")

    return None

@app.get("/manufacturers/{manufacturer_id}/cars", response_model=list[CarResponse])

```

```

async def get_manufacturer_cars(manufacturer_id: int, session: Session = Depends(get_session)):
    repo = CarRepository(session)

    cars = repo.list_all()

    filtered_cars = [car for car in cars if car.manufacturer_id == manufacturer_id]

    return filtered_cars

```

4.3. external_api.py

Асинхронні функції для роботи з `httpx.AsyncClient`, `asyncio.Task`, `gather`, `Semaphore`, `timeout` та `retry`

```

import asyncio

import httpx

async def fetch_car_with_retry_and_limit(
    client: httpx.AsyncClient,
    semaphore: asyncio.Semaphore,
    url: str,
    attempts: int = 3,
) -> dict:
    async with semaphore:
        for attempt in range(1, attempts + 1):
            try:
                response = await client.get(url)

                response.raise_for_status()

                return response.json()

            except (httpx.RequestError, httpx.HTTPStatusError):
                if attempt == attempts:
                    raise

                await asyncio.sleep(attempt)

        raise RuntimeError("Unexpected retry state")

async def fetch_group_of_cars(base_url: str, car_ids: list[int]) -> list[dict]:
    semaphore = asyncio.Semaphore(3) # обмеження паралелізму

    async with httpx.AsyncClient(timeout=5.0) as client:
        tasks = [

```

```

        asyncio.create_task(
            fetch_car_with_retry_and_limit(
                client,
                semaphore,
                f"{base_url}/cars/{car_id}"
            )
        )
    for car_id in car_ids
]

results = await asyncio.gather(*tasks)

return list(results)

```

4.4. tests/test_api.py

Автоматизовані тести для перевірки роботи API.

```

from fastapi.testclient import TestClient

from src.car_catalog.api import app

client = TestClient(app)

def test_get_cars():
    response = client.get("/cars")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_create_and_delete_car():
    response = client.post(
        "/cars",
        json={
            "make": "Toyota",
            "model": "Camry",
            "year": 2022,
            "price": 30000.0,
            "mileage": 15000,
            "vin": "1HGCR2F83HA000001",

```

```

        },
    )

    assert response.status_code == 201

    data = response.json()

    car_id = data["id"]

    assert data["make"] == "Toyota"

    # Видалення створеного авто

    del_response = client.delete(f"/cars/{car_id}")

    assert del_response.status_code == 204


def test_invalid_car_data():

    response = client.post(

        "/cars",

        json={

            "make": "Toyota",

            "model": "Camry",

            "year": 1500, # Некоректний рік (валідація Pydantic)

            "price": -100, # Некоректна ціна

            "mileage": 1000,

        },

    )

    assert response.status_code == 422

```

5. Запуск вебсервера (FastAPI)

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_7_лаб\Lab07> .venv\Scripts\python -m uvicorn src.car_catalog.api:app --reload
```

```
INFO: Will watch for changes in these directories: ['D:\\Desktop\\University\\3_1\\Professional python\\3bit_7_лаб\\Lab07']
```

```
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

```
INFO: Started reloader process [17952] using StatReload
```

```
INFO: Started server process [676]
```

```
INFO: Waiting for application startup.
```

```
INFO: Application startup complete.
```

```
INFO: 127.0.0.1:61526 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:61526 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:60912 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:60912 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [676]
INFO: Stopping reloader process [17952]
```

```
● (.venv) PS D:\Desktop\University\3_1\Professional python\3bit_7_лаб\Lab07> .venv\Scripts\python -m uvicorn src.car_catalog.api:app --reload
INFO: Will watch for changes in these directories: ['D:\\Desktop\\University\\3_1\\Professional python\\3bit_7_лаб\\Lab07']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [17952] using StatReload
INFO: Started server process [676]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:61526 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:61526 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:60912 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:60912 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [676]
INFO: Stopping reloader process [17952]
```

127.0.0.1:8000/docs

☆

Car Catalog API

1.0.0

OAS 3.1

/openapi.json

default

^

GET

/cars

Get Cars

▼

POST

/cars

Create Car

▼

GET

/cars/{car_id}

Get Car

▼

PATCH

/cars/{car_id}

Update Car

▼

DELETE

/cars/{car_id}

Delete Car

▼

GET

/manufacturers/{manufacturer_id}/cars

Get Manufacturer Cars

▼

Schemas

^

CarCreate >

Expand all

object

CarResponse >

Expand all

object

CarUpdate >

Expand all

object

HTTPValidationError >

Expand all

object

ValidationError >

Expand all

object

6. Результат

INFO: Application startup complete.

INFO: 127.0.0.1:50060 - "GET /docs HTTP/1.1" 200 OK

INFO: 127.0.0.1:50060 - "GET /openapi.json HTTP/1.1" 200 OK

INFO: 127.0.0.1:57135 - "POST /cars HTTP/1.1" 201 Created

INFO: 127.0.0.1:49915 - "GET /cars/1 HTTP/1.1" 200 OK

INFO: 127.0.0.1:58683 - "DELETE /cars/1 HTTP/1.1" 204 No Content

```
INFO: 127.0.0.1:50060 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:50060 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:57135 - "POST /cars HTTP/1.1" 201 Created
INFO: 127.0.0.1:49915 - "GET /cars/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:58683 - "DELETE /cars/1 HTTP/1.1" 204 No Content
```

7. Перевірка тестів

test_api.py

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3вїт_7_лаб\Lab07> .venv\Scripts\python -m pytest tests/test_api.py -v
```

```
===== test session starts =====
```

```
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- D:\Desktop\University\3_1\Professional
python\3вїт_7_лаб\Lab07\.venv\Scripts\python.exe
```

```
cachedir: .pytest_cache
```

```
rootdir: D:\Desktop\University\3_1\Professional python\3вїт_7_лаб\Lab07
```

```
configfile: pyproject.toml
```

```
plugins: anyio-4.15.1, asyncio-1.4.0, cov-7.1.0
```

```
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function
```

```
collected 3 items
```

```
tests/test_api.py::test_get_cars PASSED [ 33%]
```

```
tests/test_api.py::test_create_and_delete_car PASSED [ 66%]
```

```
tests/test_api.py::test_invalid_car_data PASSED [100%]
```

```
===== warnings summary =====
```

```
.venv\Lib\site-packages\fastapi\testclient.py:1
```

```
D:\Desktop\University\3_1\Professional python\3вїт_7_лаб\Lab07\.venv\Lib\site-packages\fastapi\testclient.py:1:
StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
```

```
from starlette.testclient import TestClient as TestClient # noqa
```

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
```

===== 3 passed, 1 warning in 0.67s =====

test_repositories.py

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_7_лаб\Lab07> .venv\Scripts\python -m pytest tests/test_repositories.py -v
```

===== test session starts =====

```
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- D:\Desktop\University\3_1\Professional python\3bit_7_лаб\Lab07\venv\Scripts\python.exe
```

```
cachedir: .pytest_cache
```

```
rootdir: D:\Desktop\University\3_1\Professional python\3bit_7_лаб\Lab07
```

```
configfile: pyproject.toml
```

```
plugins: anyio-4.15.1, asyncio-1.4.0, cov-7.1.0
```

```
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
```

```
collected 2 items
```

```
tests/test_repositories.py::test_crud_operations PASSED [ 50%]
```

```
tests/test_repositories.py::test_transaction_rollback PASSED [100%]
```

===== 2 passed in 0.09s =====

confest.py

```
from typing import Generator

import pytest

from sqlalchemy import create_engine

from sqlalchemy.orm import sessionmaker, Session

from application.models import Car

from application.repositories import InMemoryRepository

from src.car_catalog.database import Base

from src.car_catalog.repositories import CarDB, ManufacturerDB # noqa: F401


@pytest.fixture
def sample_cars():

    return [

        Car(make="Toyota", model="Camry", year=2020, price=25000.0, mileage=45000),

        Car(make="BMW", model="M3", year=2022, price=75000.0, mileage=12000),

        Car(make="Toyota", model="Corolla", year=2018, price=15000.0, mileage=80000),
```

```

        Car(make="Audi", model="A4", year=2020, price=28000.0, mileage=35000),
    ]

@pytest.fixture
def empty_catalog():
    return []

@pytest.fixture
def car_repository(sample_cars):
    repo = InMemoryRepository[Car]()
    for idx, car in enumerate(sample_cars):
        repo.add(f"car_{idx}", car)
    return repo

@pytest.fixture(scope="function")
def db_session() -> Generator[Session, None, None]:
    """Fixture for integration tests using SQLite in-memory database."""
    engine = create_engine("sqlite:///memory:", connect_args={"check_same_thread": False})
    Base.metadata.create_all(bind=engine)

    TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
    session = TestingSessionLocal()

    try:
        yield session
    finally:
        session.close()
        Base.metadata.drop_all(bind=engine)

```

8. Приклади JSON Request / Response

Request: POST /api/v1/cars/

```

{
    "brand": "Toyota",
    "model": "Camry",

```

```
"year": 2021,  
"price": 24500.00,  
"mileage": 45000,  
"is_available": true,  
"vin": "JTDKN3DU000123456"  
}
```

Response: Status 201 Created

```
{  
  "id": 1,  
  "brand": "Toyota",  
  "model": "Camry",  
  "year": 2021,  
  "price": 24500.0,  
  "mileage": 45000,  
  "is_available": true,  
  "vin": "JTDKN3DU000123456",  
  "created_at": "2026-10-06T16:35:00.123456"  
}
```

Request: PATCH /api/v1/cars/1

```
{  
  "price": 23000.00,  
  "mileage": 48000  
}
```

Response: Status 200 OK

```
{  
  "id": 1,  
  "brand": "Toyota",  
  "model": "Camry",  
  "year": 2021,  
  "price": 23000.0,  
  "mileage": 48000,
```

```
"is_available": true,  
"vin": "JTDKN3DU000123456",  
"created_at": "2026-10-06T16:35:00.123456"  
}
```

Request: POST /api/v1/cars/

```
{  
  "brand": "BMW",  
  "model": "X5",  
  "year": 2022,  
  "price": -500,  
  "mileage": 10000,  
  "vin": "SHORT_VIN"  
}
```

Response: Status 422 Unprocessable Entity

```
{  
  "detail": [  
    {  
      "loc": ["body", "price"],  
      "msg": "Input should be greater than 0",  
      "type": "greater_than"  
    },  
    {  
      "loc": ["body", "vin"],  
      "msg": "String should have at least 17 characters",  
      "type": "string_too_short"  
    }  
  ]  
}
```

9. Загальний опис REST API

У цьому блоці йдеться про призначення розробленого вебсервісу для керування каталогом автомобілів на базі FastAPI. Описуються його основні можливості: виконання

CRUD-операцій, фільтрація, пагінація даних, а також підтримка асинхронних процесів (паралельні запити, обробка таймаутів, ретраї).

10. Перелік маршрутів та HTTP-методів

Тут наведено повний список ендпоінтів API із відповідними HTTP-методами (GET, POST, PATCH, DELETE) та кодами відповідей (200 OK, 201 Created, 204 No Content, 404 Not Found). Описано маршрути для управління автомобілями та окремі асинхронні ендпоінти для обробки даних через HTTPX та Semaphore.

11. Приклади використання параметрів шляху та запиту

У цьому блоці пояснюється різниця між Path та Query параметрами на конкретних прикладах. Розглядається використання `car_id` у шляху URL для ідентифікації конкретного ресурсу та параметрів `brand`, `min_year`, `page`, `size` у рядку запиту для фільтрації й пагінації.

12. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git add pyproject.toml

git commit -m "build: update project configuration and dependencies"

git add src/car_catalog/models.py src/car_catalog/schemas.py src/car_catalog/repositories.py

git commit -m "feat: add car schemas, models, and repository layer"


git add src/car_catalog/main.py src/car_catalog/api.py src/car_catalog/external_api.py

git commit -m "feat: implement REST API endpoints and external async tasks"


git add tests/conftest.py tests/test_api.py

git commit -m "test: add conftest fixtures and API unit tests"
```

10.1 Результат git status

On branch main

Your branch is ahead of 'origin/main' by 4 commits.

(use "git push" to publish your local commits)

nothing to commit, working tree clean

10.2. Результат git log –online

```
b41f5b8 (HEAD -> main) test: add conftest fixtures and API unit tests
1496840 feat: implement REST API endpoints and external async tasks
ee6dfef feat: add car schemas, models, and repository layer
62f8e7a build: update project configuration and dependencies
a232942 (origin/main) Add Lab7 files
9ba9928 add cars.db in .gitignore
a0d269b feat(main): update main application execution script
b215c13 test(repositories): add unit tests and fixtures for repositories
137b98f feat(repositories): implement repository pattern for data access and transactions
c456674 feat(db): setup database configuration and alembic migrations
57fd5cf Changes for better testing
6601446 Add Lab06 files
8b2f5e2 Deleted file
a625f9a feat(app): add application layer components, services and exporters
247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration
0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock
ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests
0cd95e4 Add Lab05 files
25dd4b9 feat(models): add processing statistics models and log/output artifacts
8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint
336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter
1b19312 feat(config): add YAML application configuration and custom exception hierarchy
e6363e9 Add Lab04 files
22e99f7 Big upgrade of structure
2f4b667 feat(main): add main execution script with analytics and filter strategy
d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI
```

11. Висновок

У результаті виконання лабораторної роботи було успішно розроблено та протестовано асинхронний REST API сервіс для керування каталогом автомобілів на базі фреймворку FastAPI. Реалізований вебсервіс дає змогу виконувати повний спектр CRUD-операцій з обробкою стандартних HTTP-статусів та валідацією вхідних даних за допомогою

моделей Pydantic. Завдяки використанню асинхронного стека Python, зокрема бібліотек `asyncio` та `HTTPX`, було ефективно організовано паралельну обробку запитів, контроль конкурентності через `Semaphore` та стійкість до збоїв за допомогою таймаутів і механізму повторних спроб. Написані автоматичні `pytest`-тести підтвердили коректність функціонування розроблених ендпоінтів та шару репозиторіїв. Отримані практичні навички дозволяють ефективно проєктувати масштабовані мережеві застосунки та інтегрувати їх із сучасними базами даних.